

基于你上传的 AuraAbilitySystemGlobals 和 AuraAbilityTypes 相关文件，这里详细阐述 **GameplayEffectContext** 的用途、自定义流程、代码实现细节以及具体使用方法。

1. EffectContext 的用途及自定义目的

什么是 GameplayEffectContext (GE Context)?

FGameplayEffectContext 是 GAS 系统中用于在 GameplayEffect (GE) 之间传递非数值类“元数据”的数据包。

简单来说，当一个技能释放并应用 GE 时，Context 负责回答关于“这次打击”的背景问题：

- **Who:** 谁是始作俑者 (Instigator)? 谁是直接造成伤害的对象 (EffectCauser)?
- **Where:** 打击点在哪里 (HitResult)?
- **What:** 来源于哪个 Ability (AbilityCDO)?

为什么要自定义 Context?

默认的 FGameplayEffectContext 包含了上述基础信息，但在复杂的 RPG（如你的项目 Aura）中，这些信息远远不够。你需要传递**战斗结算后的状态**，例如：

- 是否暴击 (Is Critical Hit)?
- 是否被格挡 (Is Blocked Hit)?
- 造成了什么类型的异常状态?
- 击退的方向是什么?

这些数据通常在 ExecutionCalculation（执行计算）中得出，然后需要传递给 **UI (飘字)** 或 **GameplayCue (受击特效)**。自定义 Context 就是为了给这个数据包“扩容”，塞入这些自定义变量。

2. 自定义 Context 的标准流程

要实现自定义 Context，必须遵循以下三个标准步骤（你的代码已经完整覆盖了这三步）：

1. **定义数据结构 (AuraAbilityTypes.h/cpp):** 继承 FGameplayEffectContext，添加自定义变量，重写序列化和复制函数。
2. **重写分配器 (AuraAbilitySystemGlobals):** 告诉引擎不要创建默认的 Context，

而是创建你自定义的 Context。

3. **配置引擎 (DefaultGame.ini)**: 让引擎加载你自定义的 AbilitySystemGlobals 类 (虽然代码里没体现, 但这是必做的一步)。
-

3. 核心实现步骤与代码详解

步骤一: 定义自定义结构体 (AuraAbilityTypes.h)

这是数据载体。重点在于它必须是一个 USTRUCT, 并且支持网络序列化。

关键代码解析:

C++

```
// 必须继承自 FGameplayEffectContext
```

```
USTRUCT(BlueprintType)
```

```
struct FAuraGameplayEffectContext : public FGameplayEffectContext
```

```
{
```

```
    GENERATED_BODY()
```

```
public:
```

```
    // ... Getter/Setter ...
```

```
    // 【关键点 1】返回结构体类型信息
```

```
    // 注意: 这里通常返回 FGameplayEffectContext::StaticStruct() 以保持多态兼容性,
```

```
    // 但如果你需要在蓝图中直接转换, 有时需要特定处理。
```

```
    virtual UScriptStruct* GetScriptStruct() const
```

```
{
```

```
        return FGameplayEffectContext::StaticStruct();
```

```
}
```

```

// 【关键点 2】深拷贝 (Duplicate)

// GAS 系统会频繁复制 Context，如果不重写，自定义变量会丢失。

virtual FAuraGameplayEffectContext* Duplicate() const
{
    FAuraGameplayEffectContext* NewContext = new
FAuraGameplayEffectContext();

    *NewContext = *this; // 浅拷贝基础数据

    if (GetHitResult())
    {
        // 深拷贝 HitResult，防止悬空指针

        NewContext->AddHitResult(*GetHitResult(), true);
    }

    return NewContext;
}

// 【关键点 3】网络序列化声明

virtual bool NetSerialize(FArchive& Ar, class UPackageMap* Map, bool&
bOutSuccess);

protected:

    // 【自定义数据】这就是我们要新增的两个布尔值

    UPROPERTY()

    bool bIsBlockedHit = false;

    UPROPERTY()

    bool bIsCriticalHit = false;

};

```

// 【关键点 4】特质定义 (Traits)

// 这告诉虚幻引擎：这个结构体拥有自定义的 NetSerializer，网络传输时不要按默认方式处理，

// 而是调用上面的 NetSerialize 函数。

```
template<>
```

```
struct TStructOpsTypeTraits<FAuraGameplayEffectContext> :
```

```
TStructOpsTypeTraitsBase2<FAuraGameplayEffectContext>
```

```
{
```

```
    enum
```

```
    {
```

```
        WithCopy = true,           // 支持复制
```

```
        WithNetSerializer = true, // 支持自定义网络序列化
```

```
    };
```

```
};
```

⚠ 代码错误修正提示：

在你的 AuraAbilityTypes.h 中，我发现了一个 Bug：

C++

```
void SetIsBlockedHit(const bool InIsBlockedHit){bIsCriticalHit = InIsBlockedHit;}
```

这一行写错了！你把 bIsCriticalHit 赋值给了 BlockedHit 的 Setter。应该改为：

```
{ bIsBlockedHit = InIsBlockedHit; }
```

步骤二：实现网络序列化 (AuraAbilityTypes.cpp)

这是最复杂也最重要的一步。为了节省带宽，我们使用**位掩码 (Bitmask)** 来只复制有效的数据。

具体到单行代码解析：

C++

```

bool FAuraGameplayEffectContext::NetSerialize(FArchive& Ar, UPackageMap* Map,
bool& bOutSuccess)

{
    uint32 RepBits = 0; // 定义一个 32 位的整型，每一位代表一个变量是否存在

    // === 1. 保存模式 (Writing to Network) ===

    if (Ar.IsSaving())
    {
        // 检查父类变量是否有效，如果有效，将对应的位设为 1

        if (bReplicateInstigator && Instigator.IsValid()) { RepBits |= 1 << 0; } // 第 0 位

        // ... (省略中间部分) ...

        // 【自定义变量序列化检测】

        // 检查自定义变量。这里是强制写入位，因为 bool 值总是存在的，

        // 但为了节省空间，我们也可以判断 if (bIsBlockedHit) 再置位。

        if (bIsBlockedHit) { RepBits |= 1 << 7; } // 第 7 位代表格挡

        if (bIsCriticalHit) { RepBits |= 1 << 8; } // 第 8 位代表暴击

    }

    // === 2. 序列化掩码 ===

    // 将这个整数发给接收方，接收方就知道接下来要接收哪些数据

    Ar.SerializeBits(&RepBits, 9);

    // === 3. 读/写 数据 (Reading/Writing) ===

    // 这里的 Ar 既负责发送也负责接收（取决于 Ar.IsSaving 还是 IsLoading）

    // 如果第 0 位是 1，说明 Instigator 有数据，进行序列化

```

```

        if (RepBits & (1 << 0)) { Ar << Instigator; }

// ... (省略父类变量) ...

// 【自定义变量读写】
// 你的代码逻辑：如果位被置为 1，则读取/写入 bool
if(RepBits & (1 << 7))
{
    Ar << blsBlockedHit;
}

if(RepBits & (1 << 8))
{
    Ar << blsCriticalHit;
}

// ... (初始化逻辑) ...

return true;
}

```

逻辑总结：这段代码的核心是先通过 RepBits 告诉对方“我带了哪些行李”，然后再逐个把“行李”传过去。如果不重写这个函数，你的 blsCriticalHit 等变量在多人游戏中将无法从服务器同步到客户端。

步骤三：接管 Context 的创建 (AuraAbilitySystemGlobals)

你需要告诉 ASC (AbilitySystemComponent)：当你需要一个新的 Context 时，用我的类，别用默认的。

头文件 (AuraAbilitySystemGlobals.h):

C++

```

UCLASS()

class AURA_API UAuraAbilitySystemGlobals : public UAbilitySystemGlobals
{
    GENERATED_BODY()

    // 覆盖这个虚函数

    virtual FGameplayEffectContext* AllocGameplayEffectContext() const override;
};

```

源文件 (AuraAbilitySystemGlobals.cpp):

```

C++

FGameplayEffectContext* UAuraAbilitySystemGlobals::AllocGameplayEffectContext()
const
{
    // 【关键行】 返回 new FAuraGameplayEffectContext

    // 只要系统调用 MakeEffectContext, 这里就会创建一个包含你自定义 bool 值
    的对象。

    return new FAuraGameplayEffectContext;
}

```

4. 如何使用自定义 Context

完成上述定义后, 你需要在具体的游戏逻辑中进行**写入**和**读取**。

场景 A: 写入数据 (在 ExecutionCalculation 中)

在你的 ExecCalc_Damage 中, 你已经计算出了是否暴击/格挡, 现在要存入 Context。

```

C++

// 1. 从 ExecutionParams 获取 Context Handle

FGameplayEffectContextHandle ContextHandle = Spec.GetContext();

// 2. 将 Handle 转换为你的自定义类型指针

```

// 注意: ContextHandle 内部存的是原始指针, 我们需要拿到它并向下转型

```
FAuraGameplayEffectContext* AuraContext =
```

```
static_cast<FAuraGameplayEffectContext*>(ContextHandle.Get());
```

```
if (AuraContext)
```

```
{
```

```
    // 3. 写入数据
```

```
    AuraContext->SetIsCriticalHit(bIsCriticalHit); // 假设你算出了 bIsCriticalHit
```

```
    AuraContext->SetIsBlockedHit(bIsBlockedHit);
```

```
}
```

场景 B: 读取数据 (在 WidgetController 或 GameplayCue 中)

当伤害应用后, 你需要 UI 弹出不同颜色的数字。

C++

```
// 假设你在 WidgetController 接收到了 GameplayEffectSpec 或 ContextHandle
```

```
// 1. 获取 Context
```

```
FGameplayEffectContextHandle ContextHandle = ...;
```

```
// 2. 转换为自定义类型
```

```
const FAuraGameplayEffectContext* AuraContext = static_cast<const
```

```
FAuraGameplayEffectContext*>(ContextHandle.Get());
```

```
if (AuraContext)
```

```
{
```

```
    // 3. 读取数据
```

```
    if (AuraContext->IsCriticalHit())
```

```
    {
```

```
        // 播放暴击音效，显示金色字体
    }

    if (AuraContext->IsBlockedHit())
    {
        // 显示盾牌图标，减半伤害显示
    }
}
```

总结

1. **目的:** 为了在服务器计算伤害后，将“暴击”、“格挡”等状态同步给客户端用于表现（UI、特效）。
2. **实现:**
 - AuraAbilityTypes 定义数据结构和网络传输协议（位掩码）。
 - AuraAbilitySystemGlobals 作为一个工厂，替换了引擎默认的 Context 分配器。
3. **注意:** 务必修复 AuraAbilityTypes.h 中 SetIsBlockedHit 的 Bug，否则格挡永远无法正确被记录。